```
"""
sentiment_analyzer.py
---------------------
Sentiment-analysis logic for the AI
Sentiment Analyzer (Feature Set B).

This module uses only the Python standard
library. It works by looking up
words of a review in two dictionaries
(positive and negative words) and
turning the counts into a score between
-1.00 and +1.00.
"""

import re

# ----------------------------------------
---------------------------------
# Word dictionaries
# ----------------------------------------
---------------------------------
POSITIVE_WORDS = {
    "good", "great", "excellent",
"amazing", "awesome", "fantastic",
    "wonderful", "love", "loved", "like",
"liked", "happy", "best",
    "perfect", "enjoyable", "nice",
"useful", "helpful", "satisfied",
    "impressive", "beautiful", "brilliant",
"superb", "outstanding",
```

```
    "pleasant", "recommend", "recommended",
"fast", "reliable",
    "comfortable", "delightful",
"favorite", "favourite", "worth",
}

NEGATIVE_WORDS = {
    "bad", "terrible", "horrible", "awful",
"worst", "hate", "hated",
    "dislike", "poor", "boring", "useless",
"disappointing",
    "disappointed", "angry", "sad",
"difficult", "problem", "waste",
    "slow", "expensive", "annoying",
"broken", "defective", "rude",
    "cheap", "faulty", "unhappy",
"pathetic", "worthless", "complaint",
}

# Words that flip the meaning of the
sentiment word that follows them,
# e.g. "not good" is negative and "not bad"
is positive.
NEGATION_WORDS = {
    "not", "no", "never", "dont", "don't",
"didnt", "didn't", "isnt",
    "isn't", "wasnt", "wasn't", "cant",
"can't", "wont", "won't",
    "doesnt", "doesn't", "hardly",
"without",
}

# Classification thresholds
POSITIVE_THRESHOLD = 0.10
```

```
NEGATIVE_THRESHOLD = -0.10

EMPTY_INPUT_MESSAGE = "Please enter a
review."


# ----------------------------------------
--------------------------------
# Step 1: clean the text
# ----------------------------------------
--------------------------------
def clean_text(text):
    """Convert to lowercase and remove
punctuation (apostrophes are kept
    so that words like "don't" stay
intact)."""
    text = text.lower()
    text = text.replace("\u2019",
"'")           # curly apostrophe -> normal
    text = re.sub(r"[^a-z0-9'\s]", " ",
text)    # drop punctuation
    text = re.sub(r"\s+", " ",
text).strip()     # collapse extra spaces
    return text


# ----------------------------------------
--------------------------------
# Step 2: split into words
# ----------------------------------------
--------------------------------
def tokenize(cleaned_text):
    """Split cleaned text into a list of
words."""
```

```
    return cleaned_text.split() if
cleaned_text else []


# ----------------------------------------
---------------------------------
# Step 3: count positive and negative words
# ----------------------------------------
---------------------------------
def _is_negated(words, index):
    """True if one of the two words before
words[index] is a negation."""
    previous_words = words[max(0, index -
2):index]
    return any(word in NEGATION_WORDS for
word in previous_words)


def count_sentiment_words(words):
    """Return (positive_found,
negative_found) as two lists of words.
    A negated positive word counts as
negative and vice versa."""
    positive_found = []
    negative_found = []

    for index, word in enumerate(words):
        if word in POSITIVE_WORDS:
            if _is_negated(words, index):
                negative_found.append("not
" + word)
            else:
                positive_found.append(word)
        elif word in NEGATIVE_WORDS:
```

```
            if _is_negated(words, index):
                positive_found.append("not
" + word)
            else:
                negative_found.append(word)

    return positive_found, negative_found


# ----------------------------------------
---------------------------------
# Step 4: calculate the score
# ----------------------------------------
---------------------------------
def calculate_score(positive_count,
negative_count):
    """Normalized score in the range -1.00
to +1.00.

    score = (positive - negative) /
(positive + negative + 1)

    The "+ 1" keeps the score strictly
between -1 and +1 and stops a single
    word from giving a perfect +1.00 /
-1.00. No sentiment words -> 0.0.
    """
    total = positive_count + negative_count
    if total == 0:
        return 0.0
    return (positive_count -
negative_count) / (total + 1)
```

```
# ----------------------------------------
---------------------------------
# Step 5: classify
# ----------------------------------------
---------------------------------
def classify_sentiment(score):
    """Return 'Positive', 'Negative' or
'Neutral' for a score."""
    if score > POSITIVE_THRESHOLD:
        return "Positive"
    if score < NEGATIVE_THRESHOLD:
        return "Negative"
    return "Neutral"


# ----------------------------------------
---------------------------------
# Step 6: explanation
# ----------------------------------------
---------------------------------
def generate_explanation(positive_found,
negative_found, score, sentiment):
    """Short human-readable explanation of
the result."""
    positive_count = len(positive_found)
    negative_count = len(negative_found)

    if positive_count == 0 and
negative_count == 0:
        return ("No known positive or
negative words were found, "
                "so the review is treated
as Neutral (score 0.00).")
```

```
    positive_text = ",
".join(positive_found) if positive_found
else "none"
    negative_text = ",
".join(negative_found) if negative_found
else "none"

    if sentiment == "Positive":
        reason = f"The score is above
{POSITIVE_THRESHOLD:.2f}"
    elif sentiment == "Negative":
        reason = f"The score is below
{NEGATIVE_THRESHOLD:.2f}"
    else:
        reason = (f"The score is between
{NEGATIVE_THRESHOLD:.2f} "
                  f"and
{POSITIVE_THRESHOLD:.2f}")

    return (f"Positive words
({positive_count}): {positive_text}. "
            f"Negative words
({negative_count}): {negative_text}. "
            f"{reason}, so the review is
{sentiment}.")


# ----------------------------------------
---------------------------------
# Main function used by the GUI
# ----------------------------------------
---------------------------------
def analyze_review(review):
    """Run the full analysis. Raises
```

```
ValueError for empty input.

    Returns a dictionary with: review,
sentiment, score, explanation.
    """
    if review is None or not
review.strip():
        raise
ValueError(EMPTY_INPUT_MESSAGE)

    words = tokenize(clean_text(review))
    positive_found, negative_found =
count_sentiment_words(words)
    score =
calculate_score(len(positive_found),
len(negative_found))
    sentiment = classify_sentiment(score)
    explanation =
generate_explanation(positive_found,
negative_found,

score, sentiment)

    return {
        "review": review.strip(),
        "sentiment": sentiment,
        "score": score,
        "explanation": explanation,
    }


if __name__ == "__main__":
    # Quick console check: python
sentiment_analyzer.py
```

```
    demo = "I really loved this product. It
is excellent and amazing."
    print(analyze_review(demo))
```